

Tentu, ini adalah penjelasan **baris demi baris** untuk Rotation.cs. Kode ini dibagi menjadi dua bagian utama: **Antarmuka (UI)** dan **Matematika (Logic)**.

Saya akan memecahnya menjadi blok-blok logis agar Anda dan teman Anda lebih mudah menghafal alurnya untuk UAS.

BAGIAN 1: RequestRotation (Menangani Input User)

Fungsi ini bertugas membuat jendela *pop-up* secara manual (tanpa Visual Designer) agar user bisa memasukkan angka derajat rotasi.

C#

```
public Bitmap RequestRotation(Bitmap sourceImage)
{
    // 1. MEMBUAT JENDELA FORM BARU
    // 'using' memastikan memori Form langsung dibersihkan setelah ditutup.
    using (Form inputForm = new Form())
    {
        // Mengatur judul jendela
        inputForm.Text = "Rotasi Bebas";
        // Mengatur ukuran jendela (Lebar 300, Tinggi 160)
        inputForm.Size = new Size(300, 160);
        // Mengatur agar jendela muncul pas di tengah aplikasi induk
        inputForm.StartPosition = FormStartPosition.CenterParent;
        // Mengatur agar bingkai jendela tetap (tidak bisa ditarik-tarik ukurannya)
        inputForm.FormBorderStyle = FormBorderStyle.FixedDialog;
        // Mematikan tombol Minimize dan Maximize di pojok kanan atas
        inputForm.MinimizeBox = false;
        inputForm.MaximizeBox = false;

        // 2. MEMBUAT LABEL & TEXTBOX
        // Label tulisan "Sudut (Derajat):"
        Label lblAngle = new Label() { Left = 20, Top = 20, Text = "Sudut (Derajat):" };
        // Kotak isian angka. Default isinya "45".
        TextBox txtAngle = new TextBox() { Left = 120, Top = 20, Width = 100, Text = "45" };

        // Label info tambahan (warna abu-abu)
        Label lblInfo = new Label()
```

```

{
    Left = 20, Top = 50,
    Text = "(Positif = Searah jarum jam)",
    ForeColor = Color.Gray, // Warna teks abu-abu
    Width = 200
};

// 3. MEMBUAT TOMBOL "PUTAR"
// DialogResult.OK artinya: Kalau tombol ini diklik, Form dianggap selesai dengan status 'OK'.
Button btnOk = new Button() { Text = "Putar", Left = 100, Width = 80, Top = 80, DialogResult
= DialogResult.OK };

// 4. MENAMBAHKAN KOMPONEN KE DALAM FORM
// Kalau baris-baris ini lupa ditulis, formnya akan kosong melompong.
inputForm.Controls.Add(lblAngle);
inputForm.Controls.Add(txtAngle);
inputForm.Controls.Add(lblInfo);
inputForm.Controls.Add(btnOk);

// Mengatur agar kalau user tekan ENTER di keyboard, sama dengan klik tombol OK.
inputForm.AcceptButton = btnOk;

// 5. MENAMPILKAN FORM & MENUNGGU RESPON
// ShowDialog() akan mem-pause kode sampai user menutup jendela.
// Jika user menutup dengan tombol OK (bukan tombol Silang X)...
if (inputForm.ShowDialog() == DialogResult.OK)
{
    try
    {
        // Ubah teks di TextBox menjadi angka float.
        // float.Parse bisa error kalau user mengetik huruf, makanya pakai try-catch.
        float angle = float.Parse(txtAngle.Text);

        // Panggil Bagian 2 (Koki Utama) untuk memproses gambar
        return ProcessRotation(sourceImage, angle);
    }
    catch (Exception ex)
    {
        // Jika user mengetik huruf, tampilkan pesan error.
        MessageBox.Show("Input harus angka! Error: " + ex.Message);
    }
}
}

```

```
// Jika user batal (klik X), kembalikan null (tidak ada perubahan).
return null;
}
```

BAGIAN 2: ProcessRotation (Inti Matematika)

Ini adalah bagian terpenting. Logikanya adalah memprediksi ukuran kertas baru, lalu memutar gambar tepat di titik tengahnya.

Langkah 1: Menghitung Ukuran Baru (Bounding Box)

Kalau gambar persegi panjang diputar miring, dia butuh tempat (canvas) yang lebih luas agar ujung-ujungnya tidak terpotong.

C#

```
public Bitmap ProcessRotation(Bitmap sourceImage, float angle)
{
    // 1. SIAPKAN MATRIX SIMULASI
    Matrix matrix = new Matrix();
    // Simulasikan putaran di titik tengah gambar asli
    matrix.RotateAt(angle, new PointF(sourceImage.Width / 2, sourceImage.Height / 2));

    // 2. BUAT KOTAK BAYANGAN (GraphicsPath)
    // GraphicsPath adalah objek matematika yang merepresentasikan bentuk (shape).
    GraphicsPath path = new GraphicsPath();
    // Tambahkan kotak seukuran gambar asli ke dalam path
    path.AddRectangle(new RectangleF(0f, 0f, sourceImage.Width, sourceImage.Height));

    // Terapkan simulasi putaran tadi ke kotak bayangan
    path.Transform(matrix);

    // UKUR HASILNYA (GetBounds)
    // Komputer menghitung: "Kotak yang tadinya lurus sekarang jadi miring.
    // Berapa lebar dan tinggi total area yang dibutuhkannya sekarang?"
    RectangleF bounds = path.GetBounds();
}
```

Langkah 2: Menggambar Ulang (Re-drawing)

Setelah tahu ukuran barunya, kita buat kanvas kosong dan gambar ulang.

C#

```
// 3. BUAT BITMAP BARU (KERTAS KOSONG)
// Ukurannya diambil dari hasil perhitungan 'bounds' di atas.
Bitmap newImage = new Bitmap((int)bounds.Width, (int)bounds.Height);

// Siapkan alat lukis (Graphics) untuk menggambar di kertas baru
using (Graphics g = Graphics.FromImage(newImage))
{
    g.Clear(Color.Black); // Warnai latar belakang jadi hitam

    // Mode Interpolasi: Agar gambar yang diputar tidak pecah-pecah/bergerigi (Anti-aliasing)
    g.InterpolationMode = InterpolationMode.HighQualityBicubic;

    // 4. TRANSFORMASI KOORDINAT (RAHASIA ROTASI CENTER)
    Matrix m = new Matrix();

    // TAHAP A: Geser titik (0,0) alat lukis ke TENGAH kertas BARU
    // (bounds.Width / 2, bounds.Height / 2)
    m.Translate(newImage.Width / 2.0f, newImage.Height / 2.0f);

    // TAHAP B: Putar alat lukisnya sesuai sudut
    m.Rotate(angle);

    // TAHAP C: Tarik Mundur Gambar Asli
    // Agar titik tengah gambar asli bertemu dengan titik tengah alat lukis.
    // Geser mundur setengah lebar & tinggi gambar ASLI.
    m.Translate(-sourceImage.Width / 2.0f, -sourceImage.Height / 2.0f);

    // Terapkan matriks gabungan (A -> B -> C) ke alat lukis
    g.Transform = m;

    // Lukis gambar asli di koordinat 0,0 (yang sudah digeser-geser matriksnya)
    g.DrawImage(sourceImage, 0, 0);
}

// Kembalikan gambar hasil rotasi
return newImage;
```

}

Poin Penting untuk Dijelaskan ke Teman/Dosen:

1. Kenapa ada 3 kali Translate / Matriks?

- Jika cuma pakai `m.Rotate(angle)`, gambar akan berputar mengayun dari **pojok kiri atas (0,0)** dan sebagian besar gambar akan hilang keluar layar.
- Kombinasi **Geser Tengah Kanvas -> Putar -> Geser Tengah Gambar** memastikan gambar berputar cantik seperti kincir angin tepat di poros tengahnya (**Center Pivot**).

2. Apa fungsi `path.GetBounds()`?

- Bayangkan kertas A4 (Portrait) diputar 90 derajat jadi Landscape. Ukurannya berubah.
- Bayangkan kertas diputar 45 derajat (miring). Ukurannya jadi lebih lebar (diagonal).
- `GetBounds` menghitung otomatis ukuran kertas baru yang pas supaya **tidak ada bagian gambar yang terpotong**.

3. Kenapa pakai `InterpolationMode.HighQualityBicubic`?

- Tanpa ini, garis-garis pada gambar yang miring akan terlihat bergerigi (seperti tangga pixel kasar). Ini menghaluskan hasil rotasi.